

Contents

1. Introduction	2
2. Test Objectives	2
3. Scope of Testing	3
3.1 In Scope	3
3.2 Out of Scope	3
4. Test Environment	4
5. Test Approach	5
6. Types of Testing	5
6.1 Functional Testing	5
6.2 Positive Testing	6
6.3 Negative Testing	6
6.4 API Response Validation	6
6.5 Status Code Validation	6
6.6 Request Validation	6
6.7 Response Header Validation	6
6.8 Response Time Testing	7
6.9 Regression Testing	7
6.10 Smoke Testing	7
6.11 Status Code Testing	7
7. Test Data	7
8. Entry Criteria	8
9. Exit Criteria	8
10. Test Deliverables	9
11. Conclusion	9

Test Plan- DummyJson API Testing

1. Introduction

This test plan is prepared for the DummyJson Rest API Testing project. The purpose of this project is to verify the functionality and reliability of selected REST API endpoints using Postman.

The testing focuses on sending different API requests and validating the returned responses. It includes checking HTTP methods, status codes, response data, JSON structure, response headers, response time, and the handling of valid and invalid requests. This document defines the testing scope, objectives, test approach, test environment, test data, entry and exit criteria, risks, and testing deliverables for the project.

2. Test Objectives

The primary objective of this testing is to verify the functionality, correctness, and reliability of the selected DummyJSON REST API endpoints. The testing will focus on validating API requests and responses to ensure that the endpoints behave as expected under different conditions. The following objectives will be covered:

- Verify HTTP status codes returned by the API.
- Verify response data and JSON structure.
- Verify required request parameters and request bodies.
- Verify API behavior for valid and invalid requests.
- Verify response time is within an acceptable range.
- Identify and document defects found during testing.
- Ensure that the API behaves as expected from a user's perspective.
- Verify that API endpoints are accessible and working correctly.
- Verify HTTP methods such as GET, POST, PUT, PATCH, and DELETE where supported.

3. Scope of Testing

The scope of testing defines the areas of the DummyJSON REST API that will be covered during the testing process. The testing will focus on verifying the functionality of selected API endpoints, request and response data, HTTP status codes, and the API's behavior with different types of input.

3.1 In Scope

The following areas will be tested:

The following areas will be tested:

- Products API
- Users API
- Login API
- GET requests
- POST requests
- PUT requests where supported
- DELETE requests where supported
- Request parameters
- Request body
- Response body
- HTTP status codes
- Response headers
- Response time
- Positive testing
- Negative testing
- Basic data validation

3.2 Out of Scope

The following areas are not included in this project:

- Performance/load testing
- Security penetration testing
- Database testing
- Source-code testing
- Production environment testing
- Automated API testing

4.Test Environment

The testing was performed in a controlled environment using Postman to send requests to the DummyJSON REST API and validate the returned responses. The following environment and tools were used for test execution:

Environment Component	Details
Application Under Test(AUT)	DummyJson Rest API
API Type	RESTful API
base_url	https://dummyjson.com
Testing Tool	Postman
Testing Method	Manual API Testing
Request Methods	GET, POST, PUT, PATCH, DELETE
Response Format	JSON
Operating System	Windows
Web Browser	Google Chrome
Test Data	DummyJSON sample API data and test

	inputs
Test Environment Type	Test API

5. Test Approach

Testing will be performed manually using Postman.

The following approach will be used:

- i. Send requests to the selected API endpoints.
- ii. Verify that the request is accepted by the API.
- iii. Check the HTTP status code.
- iv. Verify the response body.
- v. Verify important response fields and data types.
- vi. Check response headers where required.
- vii. Check response time.
- viii. Test valid and invalid input values.
- ix. Record any unexpected behavior as a defect.
- x. Retest the API after fixing any identified issues, if applicable.

6. Types of Testing

The following types of testing were performed as part of the DummyJSON REST API testing project:

6.1 Functional Testing

Verify that each API endpoint performs its intended function correctly. The requests were sent to the selected endpoints and the returned responses were checked against the expected results.

6.2 Positive Testing

Positive testing was performed by sending valid requests and valid input data to verify that the API processes the requests correctly and returns the expected response.

6.3 Negative Testing

Negative testing was performed by sending invalid, incomplete, or incorrect input data to verify that the API handles invalid requests properly and returns appropriate responses.

6.4 API Response Validation

The API response body was checked to verify the JSON structure, required fields, data types, and returned values. The actual response was compared with the expected result.

6.5 Status Code Validation

HTTP status codes were verified to ensure that the API returns the appropriate status code for successful, unsuccessful, and invalid requests.

6.6 Request Validation

API requests were reviewed to verify that the required URL, parameters, query parameters, headers, and request body were correctly provided where applicable.

6.7 Response Header Validation

Response headers were checked to verify that the API returns the expected header information and content type.

6.8 Response Time Testing

The response time of the API requests was observed during testing to identify unusually slow responses under normal testing conditions.

6.9 Regression Testing

Previously tested API requests were re-tested after making changes to the test data or requests to ensure that existing functionality continued to work as expected.

6.10 Smoke Testing

Smoke testing was performed to verify that the main API endpoints were available and functioning correctly before carrying out detailed testing. Basic requests were sent to the selected endpoints to confirm that the API was responding properly, returning valid HTTP status codes, and providing a valid JSON response.

6.11 Status Code Testing

Verify that the API returns appropriate HTTP status codes such as:

- 200 – Successful request

- 201 – Resource created

- 400 – Bad request

- 404 – Resource not found

- 500 – Server error, if applicable.

7. Test Data

Test data will be created using the available DummyJSON API data and request bodies.

Examples of test data include:

- Valid product IDs
- Invalid product IDs
- Valid user IDs
- Invalid user IDs
- Valid search parameters
- Invalid search parameters
- Valid POST request data
- Invalid or incomplete request data

8. Entry Criteria

Entry criteria define the minimum conditions that must be satisfied before API testing begins. Testing will start only when the required API endpoints, testing environment, tools, test data, and test cases are available and ready for execution.

- The selected DummyJSON API endpoints are accessible and responding.
- Postman is installed and ready for API testing.
- The required API URLs and endpoints have been identified.
- Test scenarios and test cases have been prepared.
- Required valid and invalid test data is available.
- Expected results for the selected API requests are defined.
- A stable internet connection is available.
- There are no known issues preventing the execution of the planned tests.

9. Exit Criteria

Exit criteria define the conditions that must be satisfied before the API testing activities are considered complete. Testing will be concluded when the planned test cases have been executed, results have been recorded, and the major issues identified during testing have been documented.

The following conditions must be met before testing is completed:

- All planned test cases for the selected API endpoints have been executed.
- Expected and actual results have been recorded for the executed test cases.
- Functional, positive, and negative testing activities have been completed as planned.

- API response status codes, response data, JSON structure, and other required validations have been checked.
- All identified defects have been documented in the bug report.
- Critical or high-priority issues, if identified, have been reviewed and reported.
- Test execution results have been documented and summarized.

10. Test Deliverables

The following documents will be prepared as part of the project:

- Test Plan
- Test Scenarios
- Test Cases
- Bug Reports
- Test Execution Results
- Postman Collection
- Project README

11. Conclusion

This test plan provides a structured approach for testing the DummyJSON REST API using Postman. The testing focuses on API functionality, request and response validation, HTTP status codes, response data, response time, and handling of valid and invalid requests.

The objective is to identify API issues and provide clear testing documentation that demonstrates practical manual API testing skills.

